Onyx — GameKit reference

Onyx · multi-process OS on Raspberry Pi 4

*The reference of **GameKit** (`SD:/lib/gamekit.so`, `user/Kits/gamekit`): what it is for, how a program uses it, and every operation it exposes. The operations' part is made from the kit's headers by `tools/docgen/kitdocs.py` — the headers are the source. Overview of all the kits: The Kits.*

Contents

1. What it is
2. Using it
3. Index
4. `gamekit/gamekit.h`

What it is

GameKit is the games of the card's ROMs: the consoles the installed emulators play (their app.txt's `games =`), the watched folders, the ROMs found in them by their extension, and their pictures -- what the Game Library and the console's home share.

Include	`#include "gamekit/gamekit.h"`
Link	`lib/gamekit.imp.a` (C++) or `lib/gamekit.imp_c.a` (C)
Library	`SD:/lib/gamekit.so` — 9 entries in its table (`user/Kits/gamekit/gamekit.abi`, append-only)
Sources	`user/Kits/gamekit/`

Using it

What the Game Library and the console's home share about the games: **the consoles** the installed emulators play (each emulator's `app.txt`: `games = Game Boy Color: gbc; Game Boy: gb`, `order = 50` — an emulator's package installed, its console appears), **the watched folders** (`SD:/apps/gamelib.app/config.ini`, one `folder =` line each; `SD:/roms` when none is said), **the ROMs** found in them and their sub-folders by their extension — sorted by console, then by name, each with a name to show (`ZeldaOracleOfSeason.gbc` → *Zelda Oracle Of Season*) — and **their pictures**, the title screens the Game Library makes (`SD:/apps/gamelib.app/thumbs/<file>.thm`, 160 × 144). The gamepads' buttons and their settings are not in it: `Include/gamepad.h` keeps them.

Every Game Boy Advance game, its picture (C):

```
#include "gamekit/gamekit.h"

static struct game_system sys[GAMES_SYSTEMS_MAX];
static char dirs[GAMES_FOLDERS_MAX][GAMES_PATH];
static struct game g[256];
```

```
static unsigned pic[GAMES_THUMB_W * GAMES_THUMB_H];

int ns = games_systems (sys, GAMES_SYSTEMS_MAX);        // the consoles, by their order
int nd = games_folders (dirs, GAMES_FOLDERS_MAX);       // the watched folders
int n = games_scan (dirs, nd, sys, ns, g, 256);         // by console, then by name
for (int i = 0; i < n; i++)
    if (ax_streq (sys[g[i].sys].name, "Game Boy Advance"))
    {
        ax_putln (g[i].name);                           // "Star Courier"
        if (games_thumb_load (&g[i], pic)) { /* 0xRRGGBB, 160 x 144 */ }
    }
```

A game is opened with its emulator by AppKit (`lx_open (g[i].path, "")`: the emulator found by the same `app.txt`). A folder added: `games_folders_save (dirs, nd + 1)`. A program that makes the pictures saves them with `games_thumb_save`.

Index

Everything the headers declare, in their order — the details are in each header's part below.

Name	What it does	Header
`game_system`	(a type)	`gamekit.h`
`game`	(a type)	`gamekit.h`
`games_systems`	The consoles of the installed emulators (SD:/apps/*.app/app.txt with a `games` line	`gamekit.h`
`games_folders`	The watched folders (GAMES_CONFIG	`gamekit.h`
`games_folders_save`	The watched folders written (GAMES_CONFIG	`gamekit.h`
`games_system_of`	The console of a file by its extension -> its index in sys, -1 none.	`gamekit.h`
`games_nice_name`	A file's name to show	`gamekit.h`
`games_scan`	The ROMs of the folders (6 sub-folders deep at most), sorted by console (sys's order) then by name -> how many.	`gamekit.h`
`games_thumb_path`	A game's picture	`gamekit.h`
`games_thumb_load`	A game's picture	`gamekit.h`
`games_thumb_save`	A game's picture	`gamekit.h`

`gamekit/gamekit.h`

gamekit.h -- GameKit (SD:/lib/gamekit.so; docs/06-KITS-GUIDE.md "GameKit", docs/20-GAMEKIT.md): the games of the card's ROMs: the consoles the installed emulators play (each emulator's app.txt: `games`

`= Game Boy Color: gbc; Game Boy: gb`, `order = 50`), the watched folders (the Game Library's SD:/apps/gamelib.app/config.ini, one `folder = SD1:/Roms` line each; SD:/roms by default), the ROMs found in them (and their sub-folders) by their extension, and their pictures (the title screens the Game Library makes, cached in SD:/apps/gamelib.app/thumbs/.thm: 160 x 144 RGB). The Game Library and the console's shell read the same catalogue through it. C and C++: link lib/gamekit.imp.a (C++) or lib/gamekit.imp_c.a (C); the interface is append-only (gamekit.abi). (The gamepads' buttons and their settings stay in Include/gamepad.h.)

```
  struct game_system sys[GAMES_SYSTEMS_MAX]; int ns = games_systems (sys, GAMES_SYSTEMS_MAX);
  char dirs[GAMES_FOLDERS_MAX][GAMES_PATH]; int nf = games_folders (dirs, GAMES_FOLDERS_MAX);
  static struct game g[256]; int n = games_scan (dirs, nf, sys, ns, g, 256);   // by console,
then by name
  unsigned *px = new unsigned[GAMES_THUMB_W * GAMES_THUMB_H]; if (games_thumb_load (&g[0],
px)) ...
```

MIT License -- Copyright (c) 2026 Stéphane Wegener and the Onyx contributors. Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions: The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software. THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED.

```
#define GAMES_SYSTEMS_MAX   12
#define GAMES_FOLDERS_MAX   8
#define GAMES_PATH      200
#define GAMES_CONFIG        "SD:/apps/gamelib.app/config.ini"   // the watched folders
#define GAMES_THUMBS        "SD:/apps/gamelib.app/thumbs/"      // the pictures: <file>.thm
#define GAMES_THUMB_W       160
#define GAMES_THUMB_H       144

struct game_system          // a console an emulator plays
{
    char name[40];          // "Game Boy Advance"
    char ext[64];           // its files' extensions, " gba " (lower case, each between
spaces)
    char ext_text[64];      // the same to show: ".gba" (".sfc / .smc")
    char emu[24];           // its emulator: SD:/apps/<emu>.app ("gbaemu")
    int  order;         // the consoles' order (app.txt's order x 16 + its rank in `games`):
small first
};
struct game             // a ROM
{
    char path[GAMES_PATH];      // "SD1:/Roms/gba/Star_Courier.gba"
    char name[64];          // its name to show: "Star Courier"
    char key[64];           // its file's name (its picture's: GAMES_THUMBS + key + ".thm")
    int  sys;           // its console: an index in the game_system array it was scanned with
};
```

The consoles of the installed emulators (SD:/apps/*.app/app.txt with a `games` line; the emulators of before that key by their names), sorted by order -> how many (at most max).

```
int games_systems (struct game_system *out, int max);
```

The watched folders (GAMES_CONFIG; none said: SD:/roms; `folder =` with no value: none) -> how many.

```
int games_folders (char (*out)[GAMES_PATH], int max);
```

The watched folders written (GAMES_CONFIG: one `folder =` line each; n 0: none, not the default) -> 1 written.

```
int games_folders_save (const char (*folders)[GAMES_PATH], int n);
```

The console of a file by its extension -> its index in sys, -1 none.

```
int games_system_of (const char *file, const struct game_system *sys, int nsys);
```

A file's name to show: "ZeldaOracleOfSeason.gbc" -> "Zelda Oracle Of Season", "super_mario" -> "Super mario".

```
void games_nice_name (const char *file, char *out, int cap);
```

The ROMs of the folders (6 sub-folders deep at most), sorted by console (sys's order) then by name -> how many.

```
int games_scan (const char (*folders)[GAMES_PATH], int nfolders, const struct game_system
*sys, int nsys, struct game *out, int max);
```

A game's picture: its file's path (GAMES_THUMBS + key + ".thm"); read into px (GAMES_THUMB_W x GAMES_THUMB_H, 0xRRGGBB) -> 1, 0 none made yet; written from px -> 1 written.

```
void games_thumb_path (const struct game *g, char *out, int cap);
int games_thumb_load (const struct game *g, unsigned *px);
int games_thumb_save (const struct game *g, const unsigned *px);

}
#endif
```